

UVA ONLINE JUDGE — PROBLEMA 10092

The Problem with the Problem Setter

Seleção de problemas por categorias usando

Fluxo Máximo com **Edmonds-Karp**

• Grupo I

• Python 3

• Edmonds-Karp (BFS)

• Fluxo Bipartido

Contexto do Problema

O que precisa ser resolvido e por que é um problema de fluxo.



O Problema

Um elaborador de provas possui **nk categorias** e **np problemas**. Cada categoria exige uma quantidade específica de problemas para o teste.



Restrição Principal

Cada problema pode pertencer a **várias categorias**, mas só pode ser **usado uma vez**. Se atribuído a uma categoria, não pode ser reutilizado em outra.

🎯 Objetivo

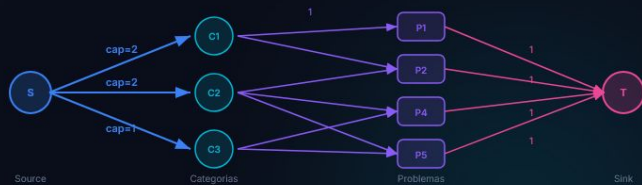
Determinar se é possível selecionar problemas que atendam a **todas** as demandas simultaneamente. Em caso positivo, mostrar **qual problema vai para qual categoria**.

💡 Por que é Fluxo Máximo?

A restrição de **uso único** + **múltiplas opções de atribuição** = **emparelhamento bipartido com demandas**, que se reduz naturalmente a uma rede de fluxo.

Modelagem da Rede de Fluxo

Como transformar o problema em uma rede com origem, sorvedouro e capacidades.



Source

Distribui demanda



S → Cat

cap = demanda[i]



Cat → Prob

cap = 1 (compat.)



Prob → Sink

cap = 1 (uso único)

Capacidades e Justificativas

Cada capacidade representa uma restrição real do enunciado.

ARESTA	CAPACIDADE	O QUE LIMITA	JUSTIFICATIVA
Source → Cat_i	demanda[i]	Quantos problemas a categoria precisa	Impõe que a categoria receba exatamente o exigido
Cat_i → Prob_j	1	Compatibilidade problema ↔ categoria	Existe se o problema pertence à categoria
Prob_j → Sink	1	Uso único de cada problema	Garante que o problema é usado no máximo uma vez

✓ Verificação da Resposta

Uma unidade de fluxo percorrendo Source → Cat_i → Prob_j → Sink significa: **"o problema j foi atribuído à categoria i"**.

Se o **fluxo máximo = soma das demandas**, todas as categorias foram atendidas.

● Origem (Source)

Representa a **necessidade total** de problemas. As arestas impõem a demanda de cada categoria.

● Sorvedouro (Sink)

Representa o **uso efetivo** de um problema. Capacidade 1 garante **uso único**.

Edmonds-Karp: Algoritmo de Fluxo

Especialização do Ford-Fulkerson usando BFS para caminhos aumentantes.

Funcionamento

- 1 **BFS** encontra o caminho mais curto de Source a Sink no **grafo residual**
- 2 Calcula o **gargalo** — menor capacidade residual no caminho
- 3 **Atualiza** o grafo residual: subtrai na direta, soma na **reversa**
- 4 Repete até **não haver mais caminho** aumentante

Por que Edmonds-Karp?

Ford-Fulkerson (DFS)

Complexidade $O(E \times f)$, onde f = valor do fluxo. Pode ser lento com capacidades grandes.

Edmonds-Karp (BFS) ✓

Complexidade $O(V \times E^2)$, independente do valor do fluxo. Mais previsível e seguro para este problema.

🔍 Arestas Reversas

Permitem **corrigir decisões anteriores**. Se um problema foi atribuído "errado", uma iteração posterior pode reverter via aresta reversa.

Execução Passo a Passo

Instância do enunciado: 3 categorias, 5 problemas — demandas: 2, 2, 1

Iteração 1

S → Cat1 → P1 → Sink

Gargalo: $\min(2,1,1) = 1$ · Fluxo: 1

Iteração 2

S → Cat1 → P4 → Sink

Gargalo: $\min(1,1,1) = 1$ · Fluxo: 2

Iteração 3

S → Cat2 → P2 → Sink

Gargalo: $\min(2,1,1) = 1$ · Fluxo: 3

Iteração 4

S → Cat2 → P5 → Sink

Gargalo: $\min(1,1,1) = 1$ · Fluxo: 4

Iteração 5

S → Cat3 → P3 → Sink

Gargalo: $\min(1,1,1) = 1$ · Fluxo: 5

Resultado

Iteração 6: BFS não encontra caminho — **PARADA**

Fluxo máximo = **5** = 2+2+1 = soma das demandas ✓

Atribuição Final

CATEGORIA	PROBLEMAS
Cat 1	Problema 1, Problema 4
Cat 2	Problema 2, Problema 5
Cat 3	Problema 3

Saída do programa

```
1
1 4
2 5
3
```

Complexidade e Casos Especiais

Análise de desempenho e tratamento de situações-limite.

Análise de Complexidade

ASPECTO	VALOR
Tempo	$O(V \cdot E^2)$
Espaço	$O(V + E)$
V (vértices)	$nk + np + 2 \leq 1.022$
E (arestas)	$O(nk \cdot np) \leq \sim 21.000$

⚡ Desempenho

Com $nk \leq 20$ e $np \leq 1000$, a execução é **rápida** e bem dentro dos limites do UVA. A estrutura dominante de memória é a **lista de arestas residuais**.

Casos Especiais

Categoria sem compatíveis

Fluxo < demanda $\rightarrow$ imprime 0

Problema multi-categoria

Múltiplas arestas Cat $\rightarrow$ Prob, mas cap=1 no Sink garante uso único

Demanda > np

Impossível: fluxo máximo $\leq np <$ demanda

Múltiplos casos de teste

Loop lê até encontrar 0 0

Conclusão

My Submissions

#	Problem	Verdict	Language	Run Time	Submission Date
31177513	10092 The Problem with the Problem Setter	Accepted	PYTH3	0.340	2026-06-12 19:27:27

● Solução Aceita

● Python 3

● $O(V \cdot E^2)$

● Grupo I

Obrigado! Alguma pergunta?